

DÉPARTEMENT D'INFORMATIQUE

LICENCE FONDAMENTAL :

GÉNIE INFORMATIQUE (GINF)

Projet de Fin d'Étude

Intitulé :

Systeme Intelligent de Détection des Infractions aux Panneaux Stop

Réalisé par :

- Abdoulaye Coulibaly
- Alfousseyni Konaté
- Judite Boli Oumar

Encadré par :

Pr. Asmae Kamal

Soutenu le 18/06/2026 devant le jury :

- **Pr. Ed-Drissiya El-Allaly, Professeur à la Faculté des Sciences - Meknès**
- **Pr. Ilham El Ouariachi, Professeur à la Faculté des Sciences - Meknès**

Année Universitaire :2025-2026



TrafficGuard



Remerciements

Au terme de ce projet de fin d'études, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à son aboutissement.

*Nos remerciements les plus sincères s'adressent avant tout à notre encadrante, **Pr. Asmae Kamal**, dont la disponibilité, la rigueur scientifique et les conseils avisés ont guidé chacune des étapes de ce travail. Sa confiance et ses encouragements nous ont permis d'aborder les difficultés techniques avec méthode et persévérance.*

Nous adressons également notre reconnaissance à l'ensemble du corps enseignant du Département d'Informatique pour la qualité de la formation dispensée tout au long de notre cursus, formation sans laquelle ce projet n'aurait pu voir le jour.

Nous remercions enfin les membres du jury qui nous font l'honneur d'évaluer ce travail, ainsi que nos familles et nos camarades de promotion pour leur soutien constant et leurs précieux encouragements.





Dédicaces

Nous dédions ce travail, fruit d'un effort commun et soutenu :

À nos parents, pour leur amour, leur patience et leur soutien indéfectible tout au long de notre parcours académique.

À nos frères, sœurs et à toutes nos familles, pour leur confiance et leurs encouragements de chaque instant.

À nos amis et à tous ceux qui nous ont accompagnés durant ces années d'études.

Et à tous les passionnés d'intelligence artificielle et d'innovation, en espérant que ce modeste travail puisse, à sa mesure, inspirer et servir.



Résumé

TrafficGuard est une application web d'analyse vidéo qui détecte automatiquement les infractions aux panneaux Stop. À partir d'une séquence filmée par une caméra fixe, le système repère les panneaux Stop, détecte et suit les véhicules image par image, puis détermine, pour chacun, s'il a réellement marqué l'arrêt obligatoire à proximité du panneau.

L'originalité du projet réside dans une distinction trop souvent négligée : détecter un objet n'est pas juger un comportement. Une infraction au Stop ne se résume pas au franchissement de l'intersection ; elle se définit par l'absence d'arrêt, un phénomène qui se mesure dans le temps. TrafficGuard s'appuie donc sur le suivi des véhicules (tracking) et sur la mesure de leur immobilité pour rendre un verdict fondé sur l'observation réelle du mouvement.

Le système repose sur le modèle de détection YOLOv8, le suiveur ByteTrack, la bibliothèque de vision OpenCV et le micro-framework web Flask, le tout déployable via Docker. Une attention particulière a été portée à la robustesse de la mesure d'arrêt : critère d'immobilité invariant à l'échelle, zone de contrôle géométriquement cohérente autour du panneau, et verdict rendu uniquement lorsque le véhicule traverse la zone du Stop. Sur la vidéo de test, le système identifie correctement le véhicule en infraction sans générer de faux positif sur un véhicule stationné hors zone.

Mots-clés : vision par ordinateur, détection d'objets, YOLOv8, suivi multi-objets, ByteTrack, sécurité routière, OpenCV, Flask.

Abstract

TrafficGuard is a web-based video-analysis application that automatically detects stop-sign violations. From footage captured by a fixed camera, the system locates stop signs, detects and tracks vehicles frame by frame, and decides, for each one, whether it actually came to the mandatory stop near the sign.

The project's core contribution lies in a frequently overlooked distinction : detecting an object is not the same as judging a behaviour. A stop-sign violation is not merely crossing the intersection ; it is defined by the absence of a stop, a phenomenon that can only be measured over time. TrafficGuard therefore relies on vehicle tracking and on measuring vehicle immobility to deliver a verdict grounded in the real observation of motion.

The system is built on the YOLOv8 detector, the ByteTrack tracker, the OpenCV vision library and the Flask web micro-framework, and can be deployed through Docker. Particular care was given to the robustness of the stop measurement : a scale-invariant immobility criterion, a geometrically consistent control zone around the sign, and a verdict issued only when the vehicle crosses the stop area. On the test video, the system correctly flags the violating vehicle while producing no false positive on a vehicle parked outside the control zone.

Keywords : computer vision, object detection, YOLOv8, multi-object tracking, ByteTrack, road safety, OpenCV, Flask.

Table des matières

Remerciements	2
Dédicaces	3
Résumé	4
Abstract	5
Introduction générale	6
1. Problématique	6
2. Solution proposée	6
3. Structure du rapport	6
Chapitre 1 : Cahier des charges	6
1.1 Présentation générale du projet.....	6
1.2 Étude de l'existant	6
1.3 Objectifs du projet	6
1.4 Méthodologie de travail	6
1.5 Acteurs et utilisateurs	6
1.6 Besoins fonctionnels et non fonctionnels.....	6
1.7 Architecture technique	6
1.8 Planification prévisionnelle.....	6
1.9 Livrables	6
1.10 Conclusion	6
Chapitre 2 : Conception et modélisation	6
2.1 Introduction	6
2.2 Diagramme de cas d'utilisation.....	6
2.3 Diagramme de classes.....	6
2.4 Diagramme de séquence	6
2.5 Architecture logicielle et pipeline de traitement.....	6
2.6 Conclusion	6
Chapitre 3 : Réalisation.....	6
3.1 Introduction	6
3.2 Environnement de développement.....	6
3.3 Le cœur algorithmique.....	6
3.4 Corrections et améliorations apportées	6
3.5 Interfaces graphiques	6
3.6 Démonstration	6
3.7 Résultats et évaluation	6
3.8 Conclusion.....	6
Conclusion générale et perspectives	6
Références	6

Table des figures

Figure 1.1 — Architecture en trois couches de TrafficGuard.....	15
Figure 1.2 — Planning prévisionnel du projet.....	15
Figure 2.1 — Diagramme de cas d'utilisation.....	17
Figure 2.2 — Diagramme de classes.....	19
Figure 2.3 — Diagramme de séquence : analyse d'une vidéo.....	20
Figure 2.4 — Pipeline de traitement d'une vidéo.....	20
Figure 3.1 — Principe de la zone de contrôle et de la mesure d'arrêt.....	23
Figure 3.2 — Interface : écran d'accueil.....	24
Figure 3.3 — Interface : analyse en cours (infraction détectée)	24
Figure 3.4 — Détection des panneaux Stop et suivi des véhicules.....	25
Figure 3.5 — Résultats de la détection et des verdicts.....	26

Liste des tableaux

Tableau 1.1 — Objectifs du projet par ordre de priorité.....	13
Tableau 1.2 — Répartition hebdomadaire des tâches.....	16
Tableau 2.1 — Cas d'utilisation : Lancer l'analyse d'une vidéo.....	18
Tableau 2.2 — Cas d'utilisation : Juger l'arrêt d'un véhicule.....	18
Tableau 3.1 — Caractéristiques des postes de travail utilisés.....	21
Tableau 3.2 — Classes COCO exploitées par TrafficGuard.....	22
Tableau 3.3 — Principaux réglages de calibrage du système.....	23
Tableau 3.4 — Corrections et améliorations apportées au moteur.....	24

Introduction générale

La sécurité routière constitue l'un des enjeux majeurs des sociétés contemporaines. Chaque année, un nombre considérable d'accidents survient aux intersections, et une part importante d'entre eux résulte du non-respect des règles de priorité, au premier rang desquelles figure le panneau Stop. Ce panneau impose une règle simple mais déterminante : tout véhicule doit marquer un arrêt complet avant de franchir l'intersection, afin de céder le passage et de s'assurer que la voie est libre. Le non-respect de cette obligation est une cause fréquente de collisions, parfois graves.

La surveillance de cette règle repose aujourd'hui essentiellement sur la présence physique d'agents, une approche à la fois coûteuse, difficile à généraliser et soumise aux limites de l'attention humaine. Or, les progrès récents de l'intelligence artificielle, et plus particulièrement de la vision par ordinateur, ouvrent une alternative crédible : analyser automatiquement des séquences vidéo afin de repérer les comportements infractionnels. Les modèles de détection d'objets atteignent désormais une précision et une rapidité qui rendent envisageable une surveillance automatisée des intersections.

C'est dans ce contexte que s'inscrit notre projet de fin d'études, TrafficGuard : une application web capable de détecter les panneaux Stop, de suivre les véhicules dans la scène, et de déterminer si chacun a effectué l'arrêt obligatoire. Le présent rapport décrit la démarche suivie, les choix techniques retenus, l'architecture du système, sa réalisation, ainsi que les résultats obtenus et les limites — présentées de manière honnête et assumée.

1. Problématique

La surveillance manuelle des intersections équipées de panneaux Stop présente plusieurs limites structurelles. Elle mobilise un personnel important pour couvrir ne serait-ce qu'un seul carrefour en continu ; elle ne peut être généralisée à grande échelle ; elle est sujette à la fatigue et à la subjectivité de l'observateur ; et elle ne produit aucune donnée exploitable a posteriori.

Au-delà de ces limites pratiques, une difficulté technique centrale doit être soulignée, car elle conditionne toute la conception du système. Détecter une infraction à un panneau Stop ne se réduit pas à constater qu'un véhicule franchit l'intersection. L'infraction consiste précisément à NE PAS s'arrêter. Or, l'arrêt est un phénomène qui se déroule dans le temps : pour le constater, il faut observer le mouvement d'un véhicule sur une séquence d'images successives, et non sur une image isolée. Un système qui se contenterait de repérer un véhicule à proximité d'un panneau le déclarerait en infraction même s'il s'est parfaitement immobilisé — ce qui n'a aucun sens.

La problématique centrale de ce projet peut donc se formuler ainsi : comment détecter, de manière automatique et fiable, les véhicules qui ne respectent pas l'arrêt obligatoire à un panneau Stop, en s'appuyant sur l'analyse vidéo par intelligence artificielle ?

2. Solution proposée

Pour répondre à cette problématique, nous proposons TrafficGuard, une application web qui combine détection d'objets, suivi multi-objets et mesure du mouvement. Plutôt que de supposer qu'un simple passage constitue une infraction, le système mesure véritablement l'immobilité de chaque véhicule à proximité du panneau. Ses caractéristiques distinctives sont les suivantes :

- Détection réelle de l'infraction : le système mesure l'arrêt dans le temps, au lieu de déduire l'infraction d'un simple franchissement.
- Suivi fiable des véhicules : chaque véhicule reçoit un identifiant stable, ce qui évite tout double comptage et permet de reconstituer sa trajectoire.
- Zone de contrôle cohérente : une région d'intérêt est définie autour du panneau Stop ; le verdict n'est rendu que lorsque le véhicule la traverse.
- Interface web moderne : l'application s'utilise depuis un simple navigateur, sans installation, avec un suivi en temps réel des compteurs et de l'historique.
- Double mode d'analyse : un mode vidéo pour le jugement complet des infractions, et un mode image pour la détection seule.
- Déploiement reproductible : grâce à Docker, l'application peut être mise en ligne sur des plateformes telles que Hugging Face Spaces ou exécutée sur Google Colab.

3. Structure du rapport

Ce rapport est organisé de manière progressive afin de refléter les étapes de conception et de réalisation du projet :

- Le chapitre 1, Cahier des charges, définit le périmètre du projet : présentation générale, objectifs, méthodologie, acteurs, besoins fonctionnels et non fonctionnels, contraintes, planification et livrables.
- Le chapitre 2, Conception et modélisation, présente la modélisation UML (cas d'utilisation, classes, séquence) ainsi que l'architecture logicielle et le pipeline de traitement.
- Le chapitre 3, Réalisation, décrit l'environnement de développement, le cœur algorithmique, les corrections apportées pour fiabiliser la mesure d'arrêt, les interfaces, la démonstration et les résultats obtenus.
- La conclusion générale dresse le bilan du travail accompli et ouvre sur les perspectives d'évolution.

Chapitre 1 : Cahier des charges

La réalisation d'une application combinant intelligence artificielle et développement web nécessite un cadrage précis dès les premières étapes. Ce chapitre traduit les besoins du projet en exigences fonctionnelles, techniques et organisationnelles. Il constitue la colonne vertébrale du travail en assurant la cohérence entre les objectifs visés, les choix techniques et les contraintes à respecter.

1.1 Présentation générale du projet

TrafficGuard est une application web destinée à l'analyse automatique du respect des panneaux Stop. Elle s'adresse à un opérateur (agent de sécurité routière, technicien ou chercheur) qui souhaite analyser des séquences vidéo d'intersections afin d'identifier les véhicules en infraction et d'obtenir des statistiques objectives.

Le système fonctionne à partir d'une vidéo issue d'une caméra fixe filmant une intersection. Il détecte les panneaux Stop, suit les véhicules dans le champ de la caméra et juge, pour chacun, le respect de l'arrêt obligatoire. Un mode complémentaire permet d'analyser une image fixe (détection seule, sans jugement d'infraction). Une extension future pourrait étendre le système à d'autres types d'infractions (franchissement de feu rouge, excès de vitesse).

1.2 Étude de l'existant

Avant de concevoir TrafficGuard, nous avons examiné les approches existantes de surveillance du respect des panneaux Stop, afin de situer notre solution et d'en justifier les choix.

- La surveillance humaine, assurée par des agents sur le terrain, reste la méthode la plus répandue. Fiable sur de courtes durées, elle est coûteuse, non généralisable et ne produit aucune donnée exploitable.
- Les systèmes de radars et de caméras automatiques sont surtout déployés pour les excès de vitesse et les feux rouges. Ils reposent généralement sur des capteurs dédiés (boucles au sol, radars) et ne mesurent pas l'arrêt à un Stop, qui suppose d'analyser le mouvement d'un véhicule dans le temps.
- Les solutions de vision par ordinateur existantes se concentrent le plus souvent sur la détection ou le comptage de véhicules, sans juger un comportement réglementaire précis comme l'arrêt obligatoire.

Il ressort de cette étude qu'aucune solution simple et accessible ne mesure véritablement l'arrêt à un panneau Stop à partir d'une vidéo. TrafficGuard se positionne précisément sur ce besoin : juger un comportement (l'arrêt) plutôt que constater une présence, à l'aide d'outils libres et d'une caméra ordinaire.

1.3 Objectifs du projet

Les objectifs du projet, classés par ordre de priorité, sont synthétisés dans le tableau 1.1.

N°	Objectif	Priorité
1	Détecter les panneaux Stop dans une vidéo de circulation	Haute
2	Détecter et suivre les véhicules image par image	Haute
3	Mesurer si un véhicule marque un arrêt à proximité du Stop	Haute
4	Signaler les véhicules en infraction (absence d'arrêt)	Haute
5	Comptabiliser distinctement infractions et arrêts respectés	Haute
6	Fournir un historique horodaté des événements	Moyenne
7	Analyser une image fixe (détection seule, sans jugement)	Moyenne
8	Proposer une interface web moderne et accessible	Haute

Tableau 1.1 — Objectifs du projet par ordre de priorité

1.4 Méthodologie de travail

1.4.1 Organisation de l'équipe

Le projet a été mené en équipe selon une répartition claire des responsabilités, avec une coordination régulière. Un membre s'est concentré sur le moteur de détection (intégration de YOLOv8 et de ByteTrack, logique de mesure de l'arrêt), tandis qu'un autre a pris en charge l'interface web et l'intégration côté serveur (Flask, routes, affichage temps réel). Des points d'avancement réguliers ont permis d'ajuster les priorités et de résoudre rapidement les difficultés techniques rencontrées.

1.4.2 Technologies retenues

Les technologies ont été choisies pour leur adéquation au problème, leur maturité et leur caractère libre :

- **Python** — langage de référence de l'intelligence artificielle, utilisé pour l'ensemble du traitement et du serveur.
- **YOLOv8** — modèle de détection d'objets en temps réel (panneaux et véhicules).
- **ByteTrack** — algorithme de suivi multi-objets attribuant un identifiant stable à chaque véhicule.
- **OpenCV** — lecture des vidéos, traitement et annotation des images.
- **NumPy** — calculs numériques rapides (déplacements, distances).
- **Flask** — micro-framework web servant de pont entre le moteur et le navigateur.
- **HTML, CSS, JavaScript** — interface web (thème sombre, mise à jour temps réel).
- **Docker** — empaquetage et déploiement reproductible de l'application.

1.5 Acteurs et utilisateurs

Le système met en jeu un acteur principal unique, l'utilisateur (opérateur), dont les fonctionnalités attendues sont les suivantes :

- Charger une vidéo ou une image à analyser depuis son navigateur.
- Lancer, mettre en pause, reprendre ou arrêter l'analyse d'une vidéo.
- Visualiser en temps réel la vidéo annotée, les compteurs et l'historique des événements.
- Consulter le verdict rendu pour chaque véhicule (infraction ou arrêt respecté).
- Analyser une image fixe pour vérifier la détection des panneaux et des véhicules.

1.6 Besoins fonctionnels et non fonctionnels

1.6.1 Besoins fonctionnels

- Détection des panneaux Stop et des véhicules (voiture, moto, bus, camion).
- Suivi des véhicules avec identifiant stable et reconstitution de leur trajectoire.
- Mesure de l'immobilité d'un véhicule dans la zone de contrôle du panneau.
- Décision d'infraction au franchissement de la zone, avec jugement unique par véhicule.
- Comptage distinct des infractions et des arrêts respectés, et historique horodaté.
- Affichage temps réel et fonctions de pause/reprise/arrêt.

1.6.2 Besoins non fonctionnels et contraintes

- Fiabilité : la mesure de l'arrêt doit être robuste aux variations d'échelle et de résolution.
- Accessibilité : l'application doit s'utiliser depuis un simple navigateur, sans installation.
- Performance : le traitement doit rester fluide ; il s'exécute dans un thread séparé pour ne pas bloquer le serveur.
- Portabilité : l'application doit pouvoir s'exécuter en local, sur Google Colab et en ligne via Docker.
- Honnêteté méthodologique : les limites du système (caméra fixe, conditions d'éclairage) doivent être explicitement documentées.

1.7 Architecture technique

L'application repose sur une architecture en trois couches qui sépare clairement les responsabilités : une couche de présentation (navigateur), une couche applicative (serveur Flask) et une couche de traitement (moteur de détection). Cette organisation, détaillée au chapitre 2, garantit la maintenabilité et la lisibilité du système (figure 1.1).

Architecture en trois couches

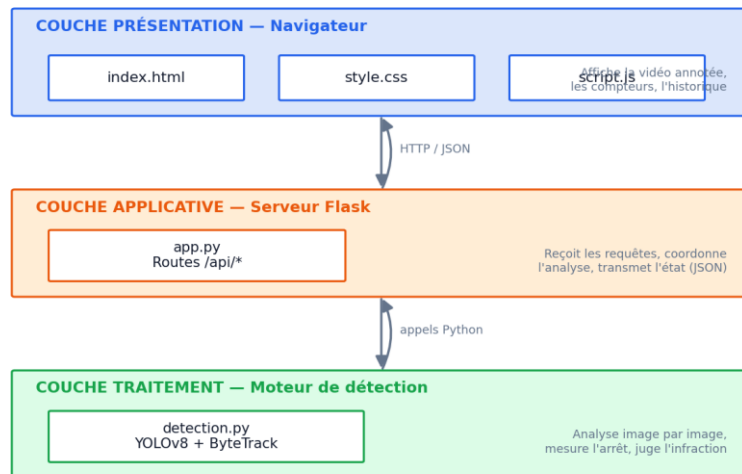


Figure 1.1 — Architecture en trois couches de TrafficGuard

1.8 Planification prévisionnelle

Le projet a été planifié sur une dizaine de semaines, depuis l'étude du sujet jusqu'à la rédaction du rapport et la préparation de la soutenance. La figure 1.2 et le tableau 1.2 en présentent le déroulement.

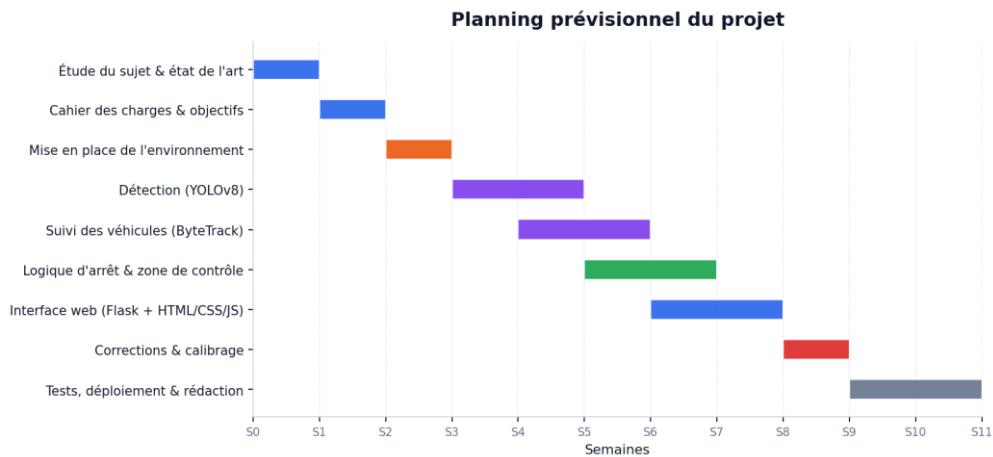


Figure 1.2 — Planning prévisionnel du projet

Semaine	Tâches réalisées
S0–S1	Étude du sujet, état de l'art, rédaction du cahier des charges et des objectifs
S2	Mise en place de l'environnement (Python, dépendances, structure du projet)
S3–S4	Détection des panneaux et des véhicules avec YOLOv8 ; intégration du suivi ByteTrack
S5	Conception de la logique d'arrêt et de la zone de contrôle
S6–S7	Développement de l'interface web (Flask, HTML/CSS/JS) et de l'affichage temps réel
S8	Corrections, calibrage et fiabilisation de la mesure d'arrêt

Semaine	Tâches réalisées
S9–S10	Tests, déploiement (Docker/Colab), rédaction du rapport et préparation de la soutenance

Tableau 1.2 — Répartition hebdomadaire des tâches

1.9 Livrables

- Le code source complet de l'application (moteur de détection, serveur et interface).
- Le présent rapport de projet de fin d'études.
- Une présentation de soutenance synthétisant la démarche et les résultats.
- Un guide d'installation et d'utilisation (exécution locale, Colab et déploiement Docker).
- Une démonstration en temps réel illustrant les deux verdicts (infraction et arrêt respecté).

1.10 Conclusion

Ce chapitre a posé le cadre du projet en précisant ses objectifs, ses acteurs, ses besoins et ses contraintes. Le chapitre suivant traduit ces exigences en une conception structurée, à travers la modélisation UML et la définition de l'architecture logicielle.

Chapitre 2 : Conception et modélisation

2.1 Introduction

Avant la réalisation technique, une phase de conception est indispensable pour formaliser les besoins, structurer le système et clarifier les interactions entre ses composants. Ce chapitre présente la modélisation du projet à l'aide du langage UML (Unified Modeling Language), à travers les diagrammes de cas d'utilisation, de classes et de séquence. Il décrit ensuite l'architecture logicielle en couches et le pipeline de traitement qui en découle.

2.2 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation offre une vue globale des interactions entre l'utilisateur et le système. Il met en évidence les fonctionnalités majeures et les dépendances entre elles.

2.2.1 Acteur du système

Le système ne comporte qu'un acteur principal : l'utilisateur (opérateur). Il charge un média, pilote l'analyse et consulte les résultats. Les traitements internes — détection, suivi, mesure et jugement — sont des cas d'utilisation inclus, déclenchés automatiquement par le lancement de l'analyse, comme l'illustre la figure 2.1.

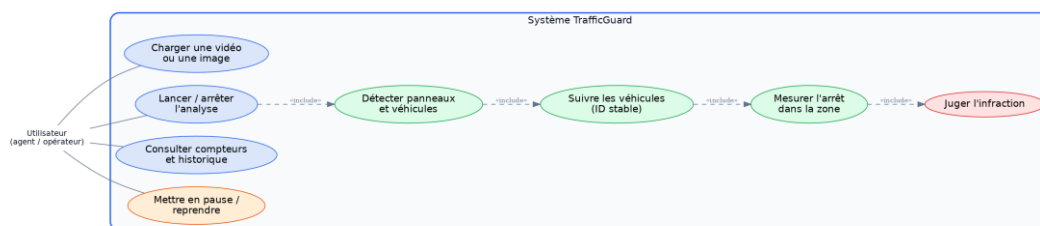


Figure 2.1 — Diagramme de cas d'utilisation

2.2.2 Description des cas d'utilisation

Les tableaux 2.1 et 2.2 détaillent textuellement deux cas d'utilisation représentatifs, en précisant acteurs, préconditions, postconditions et scénarios.

Cas d'utilisation	Lancer l'analyse d'une vidéo
Acteur	Utilisateur
Objectif	Analyser une vidéo afin d'identifier les véhicules en infraction
Préconditions	Une vidéo valide a été chargée ; aucune analyse n'est déjà en cours
Postconditions	L'analyse est lancée dans un thread ; compteurs et image annotée sont mis à jour
Scénario nominal	1. L'utilisateur sélectionne une vidéo et clique sur « Lancer ». 2. Le navigateur envoie le fichier puis la commande de démarrage au serveur. 3. Le serveur lance l'analyse dans un thread séparé. 4. L'interface s'actualise toutes les 250 ms (vidéo annotée, compteurs, historique).

Cas d'utilisation	Lancer l'analyse d'une vidéo
Scénario alternatif	A1 — Une analyse est déjà en cours : le système refuse le lancement et invite à l'arrêter d'abord.

Tableau 2.1 — Cas d'utilisation : Lancer l'analyse d'une vidéo

Cas d'utilisation	Juger l'arrêt d'un véhicule
Acteur	Système (cas inclus)
Objectif	Déterminer si un véhicule a respecté l'arrêt obligatoire au Stop
Préconditions	Un panneau Stop est détecté ; le véhicule est suivi et a pénétré la zone de contrôle
Postconditions	Un verdict unique (infraction ou respecté) est attribué au véhicule
Scénario nominal	1. Le véhicule entre dans la zone de contrôle du panneau. 2. Le système mesure son immobilité (déplacement relatif à sa taille). 3. À la sortie de la zone, le système rend son verdict. 4. Si le véhicule s'est immobilisé : arrêt respecté ; sinon : infraction.
Scénario alternatif	A1 — Le véhicule disparaît (occlusion, bord de l'image) après être entré dans la zone : le verdict est rendu sur la base de l'historique disponible.

Tableau 2.2 — Cas d'utilisation : Juger l'arrêt d'un véhicule

2.3 Diagramme de classes

Le diagramme de classes décrit la structure interne du système (figure 2.2). Trois classes principales se dégagent :

- **DetecteurVideo** — le moteur central. Elle charge le modèle YOLO, lit la vidéo image par image, détecte et suit les objets, calcule la zone de contrôle, rend les verdicts et expose l'état courant de l'analyse.
- **Vehicule** — représente un véhicule suivi. Elle conserve l'historique des positions et des tailles de sa boîte englobante, l'état de son arrêt, son verdict et l'indicateur indiquant s'il a déjà été jugé.
- **Application Flask** — la couche serveur. Elle expose les routes de l'application et délègue les traitements à une instance unique de DetecteurVideo.

La classe DetecteurVideo gère une collection de Vehicule (relation un-à-plusieurs) et utilise la bibliothèque YOLO pour la détection et le suivi.

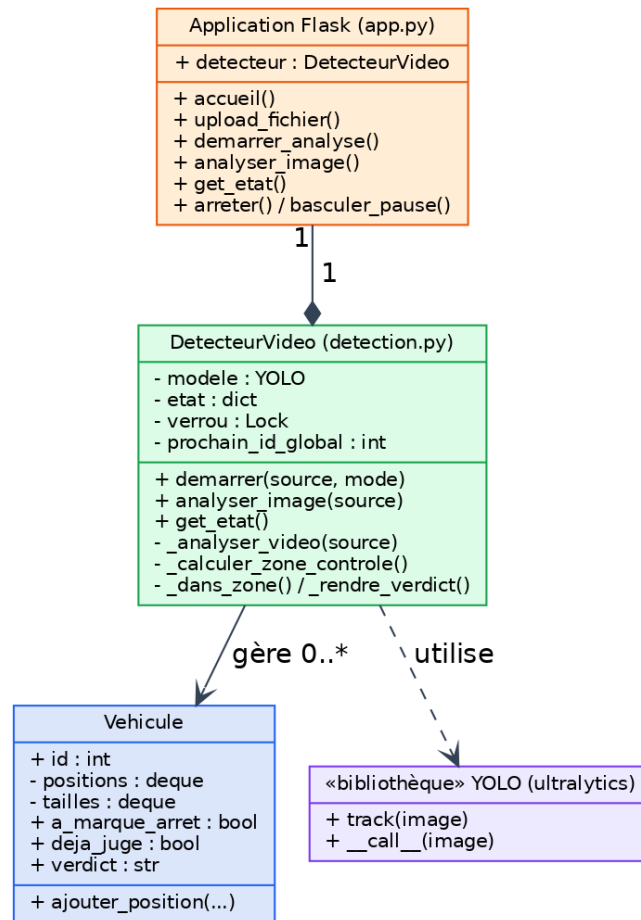


Figure 2.2 — Diagramme de classes

2.4 Diagramme de séquence

Le diagramme de séquence de la figure 2.3 illustre le déroulement chronologique de l'analyse d'une vidéo, depuis le chargement du fichier jusqu'à l'affichage des verdicts. Deux flux coexistent : le flux d'analyse, exécuté en arrière-plan dans un thread, et le flux d'actualisation, déclenché périodiquement par le navigateur (polling) toutes les 250 millisecondes. Cette séparation permet au serveur de rester réactif pendant que l'analyse, potentiellement longue, se poursuit.

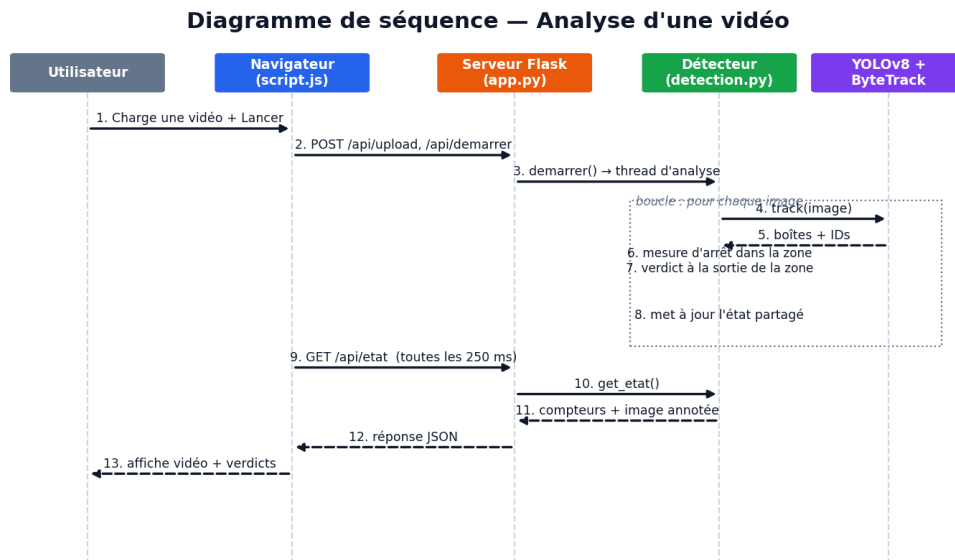


Figure 2.3 — Diagramme de séquence : analyse d'une vidéo

2.5 Architecture logicielle et pipeline de traitement

L'application est structurée en trois couches communicantes (présentation, applicative, traitement), déjà introduites au chapitre 1. La couche de traitement met en œuvre un pipeline en cinq étapes : lecture des images, détection des objets, suivi des véhicules, mesure de l'arrêt dans la zone de contrôle, et décision du verdict (figure 2.4).

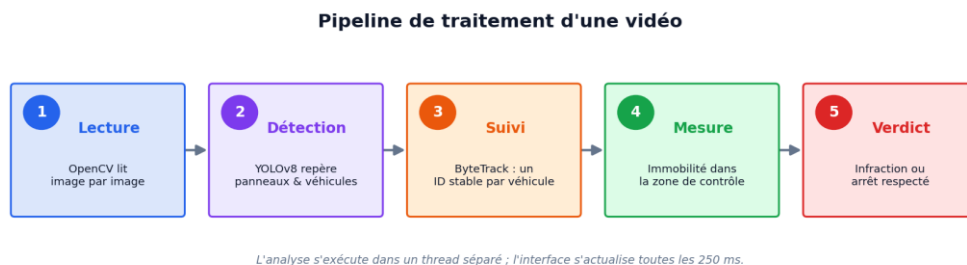


Figure 2.4 — Pipeline de traitement d'une vidéo

2.6 Conclusion

La modélisation présentée dans ce chapitre a permis de formaliser les besoins fonctionnels, de structurer le système et de clarifier ses interactions internes. Ce socle conceptuel guide la réalisation technique décrite dans le chapitre suivant.

Chapitre 3 : Réalisation

3.1 Introduction

Ce chapitre retrace la mise en œuvre technique de TrafficGuard : l'environnement de développement, le cœur algorithmique, les corrections apportées pour fiabiliser la mesure d'arrêt, les interfaces réalisées, la démonstration sur une vidéo réelle et, enfin, les résultats obtenus.

3.2 Environnement de développement

3.2.1 Environnement matériel

Le développement et les tests se sont appuyés sur des machines personnelles, ainsi que sur l'environnement Google Colab pour bénéficier d'une accélération GPU lors des analyses.

Poste	Caractéristiques
Ordinateur portable	Processeur multicœur, 8–16 Go de RAM, SSD, système Windows / Linux
Google Colab (T4 GPU)	GPU NVIDIA T4, environnement Python prêt à l'emploi, exécution dans le navigateur

Tableau 3.1 — Caractéristiques des postes de travail utilisés

3.2.2 Environnement logiciel

Les principaux outils et bibliothèques mobilisés sont les suivants :

- **Python 3.11** — langage principal du projet.
- **Ultralytics YOLOv8 (yolov8n)** — détection d'objets ; la variante « nano », la plus légère, offre un bon compromis vitesse/précision et fonctionne sans matériel spécialisé.
- **ByteTrack** — suiveur multi-objets intégré à YOLOv8, garantissant un identifiant stable par véhicule.
- **OpenCV** — lecture vidéo, redimensionnement, rotation, annotation et encodage des images.
- **NumPy** — manipulation efficace des matrices de pixels et calcul des déplacements.
- **Flask + Gunicorn** — serveur web de développement et serveur de production.
- **Docker** — conteneurisation pour un déploiement reproductible (Hugging Face Spaces).
- **Visual Studio Code & Git** — édition du code et gestion de versions.

3.2.3 Déploiement

TrafficGuard a été pensé pour être facilement exécutable et déployable. Trois modes de mise en œuvre sont possibles. En local, l'installation des dépendances (fichier requirements.txt) puis le lancement du serveur Flask suffisent à ouvrir l'application dans un navigateur. Sur Google Colab, l'activation d'un GPU T4 accélère l'analyse ; un tunnel ngrok expose alors l'interface sur une adresse publique temporaire. Enfin, le projet inclut un Dockerfile décrivant,

étape par étape, la construction d'un conteneur reproductible, ce qui permet un déploiement en ligne sur des plateformes telles que Hugging Face Spaces, sur le port 7860.

3.3 Le cœur algorithmique

Le moteur de détection (fichier `detection.py`) constitue le cœur du système. Son fonctionnement s'articule autour de quatre opérations enchaînées pour chaque image de la vidéo.

3.3.1 Détection des objets

YOLOv8 analyse chaque image et renvoie, pour chaque objet détecté, une boîte englobante, une classe et un score de confiance. TrafficGuard exploite un sous-ensemble des classes du jeu de données COCO sur lequel le modèle a été entraîné (tableau 3.2). Le panneau Stop sert à définir la zone de contrôle ; les véhicules sont suivis et jugés.

Classe COCO	Objet	Rôle dans TrafficGuard
2	Voiture	Suivi et jugement
3	Moto	Suivi et jugement
5	Bus	Suivi et jugement
7	Camion	Suivi et jugement
11	Panneau Stop	Définition de la zone de contrôle

Tableau 3.2 — Classes COCO exploitées par TrafficGuard

3.3.2 Suivi des véhicules

La détection seule ne suffit pas : il faut reconnaître qu'un véhicule présent sur une image est le même que celui de l'image précédente. Cette tâche, le suivi (tracking), est assurée par ByteTrack. Pour éviter le recyclage des identifiants lorsqu'un véhicule disparaît brièvement, le système associe à chaque identifiant de ByteTrack un identifiant global stable, conservé tout au long de la présence du véhicule.

3.3.3 Mesure de l'arrêt et zone de contrôle

C'est l'élément déterminant du projet. Le système définit autour du panneau Stop une zone de contrôle : une région rectangulaire dont les dimensions sont proportionnelles à la taille du panneau détecté. Tant qu'un véhicule se trouve dans cette zone, le système mesure son immobilité. Le verdict est rendu lorsque le véhicule sort de la zone — autrement dit, lorsqu'il a franchi l'aire du Stop (figure 3.1).

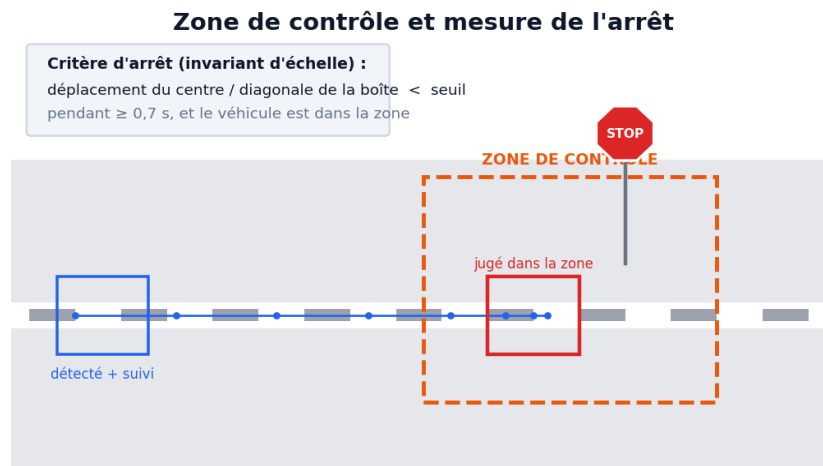


Figure 3.1 — Principe de la zone de contrôle et de la mesure d'arrêt

Le tableau 3.3 récapitule les principaux réglages de calibrage, regroupés en tête du moteur pour faciliter l'adaptation du système à différentes scènes.

Réglage	Rôle	Valeur
SEUIL_CONFIANCE	Certitude minimale pour accepter une détection YOLO	0,35
RATIO_ARRET	Déplacement maximal (rapporté à la taille du véhicule) pour être jugé immobile	0,020
DUREE_ARRET_MIN	Durée d'immobilité requise pour valider un arrêt	0,7 s
ZONE_FACTEUR_*	Dimensions de la zone de contrôle en multiples de la taille du panneau	6× / 7×
MEMOIRE_STOP	Mémorisation du panneau s'il disparaît brièvement	90 images

Tableau 3.3 — Principaux réglages de calibrage du système

3.4 Corrections et améliorations apportées

Une première version fonctionnelle du moteur présentait plusieurs fragilités qui pouvaient fausser les verdicts. Une revue critique du code a conduit à six corrections majeures, résumées dans le tableau 3.4. Ces améliorations constituent un apport important du projet : elles transforment une démonstration de principe en un système dont la décision est géométriquement et physiquement cohérente.

Réf.	Problème de la version initiale	Correction apportée
C1	L'immobilité était mesurée en pixels bruts, donc dépendante de la distance et de la résolution	Critère invariant à l'échelle : déplacement rapporté à la diagonale de la boîte du véhicule

Réf.	Problème de la version initiale	Correction apportée
C2	La ligne de jugement était placée sur le pixel même du panneau, ce qui est géométriquement incohérent	Zone de contrôle rectangulaire (ROI) proportionnelle à la taille du panneau
C3	Un véhicule immobilisé n'importe où à l'écran était compté comme « arrêt respecté »	L'immobilité n'est comptabilisée que lorsque le véhicule est DANS la zone du Stop
C4	La logique de franchissement, séparée en cas horizontal/vertical, était fragile	Verdict unifié rendu à la sortie de la zone, valable dans toutes les directions
C5	Le bruit de détection provoquait de faux arrêts ou de faux mouvements	Lissage de la trajectoire sur plusieurs positions successives
C6	Les réglages étaient dispersés et exprimés en valeurs absolues	Paramètres regroupés et exprimés en proportions, plus faciles à calibrer

Tableau 3.4 — Corrections et améliorations apportées au moteur

3.5 Interfaces graphiques

L'interface adopte un thème sombre moderne. Elle se compose d'une zone principale, à gauche, affichant la vidéo annotée et les commandes, et d'un panneau latéral, à droite, regroupant l'état du panneau Stop, les compteurs (infractions, arrêts respectés, véhicules suivis) et l'historique des derniers événements horodatés.

Avant toute analyse, l'écran d'accueil invite l'utilisateur à charger un média et à choisir le mode (vidéo ou image), comme le montre la figure 3.2.

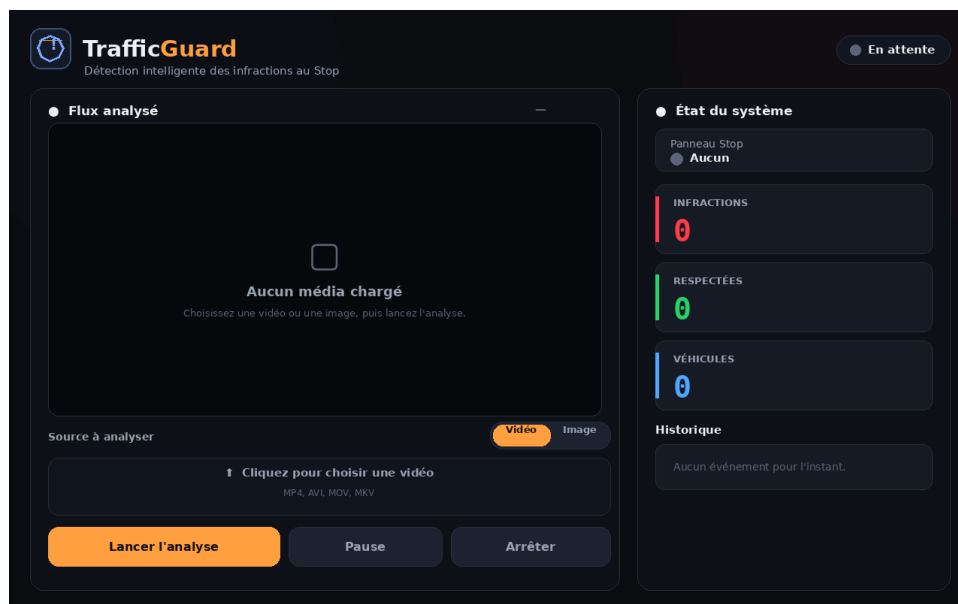


Figure 3.2 — Interface : écran d'accueil

Pendant l'analyse, la vidéo annotée s'affiche en temps réel ; les compteurs évoluent, l'historique s'enrichit et une bannière d'alerte apparaît à chaque infraction (figure 3.3).

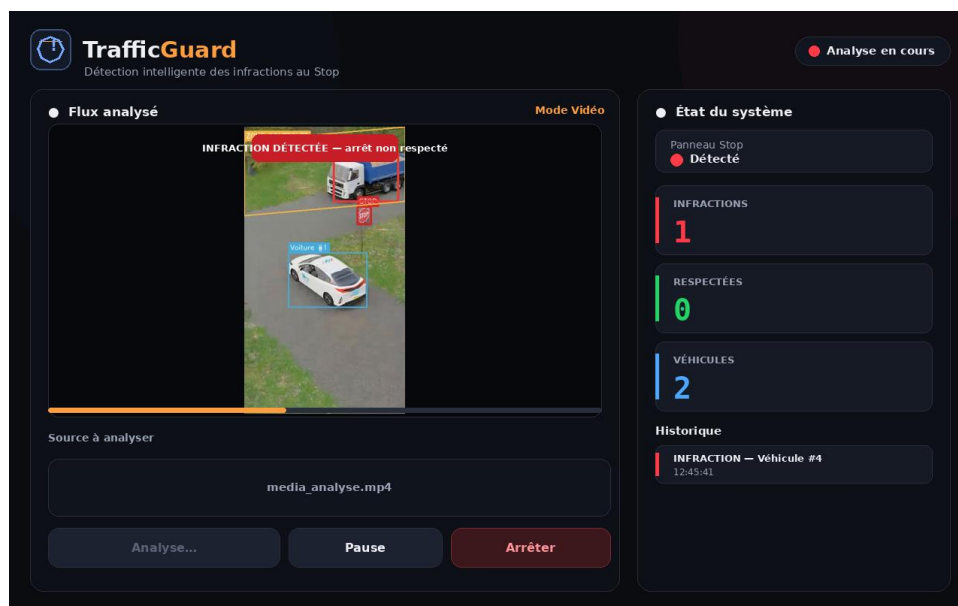


Figure 3.3 — Interface : analyse en cours (infraction détectée)

3.6 Démonstration

Le système a été éprouvé sur une vidéo de test filmée par une caméra fixe surplombant une intersection munie d'un panneau Stop. Le système y détecte le panneau, suit les véhicules et rend un verdict pour chacun ; la figure 3.4 illustre l'étape de détection et de suivi.

Dans un premier temps, le système détecte le panneau Stop (cadre rouge) ainsi que les véhicules présents (cadres bleus), auxquels il attribue un identifiant (figure 3.4).

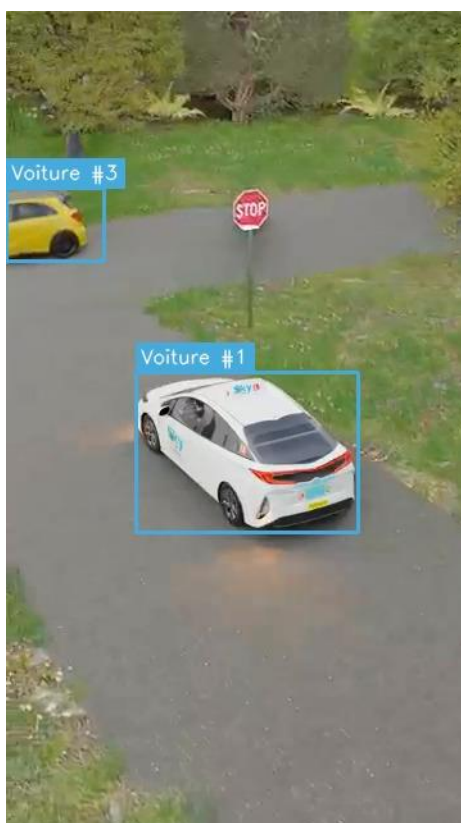


Figure 3.4 — Détection des panneaux Stop et suivi des véhicules

Lors de la démonstration en direct, l'analyse complète de la vidéo de test est présentée : les véhicules sont suivis tout au long de leur passage et, pour chacun, le système affiche le verdict correspondant — « arrêt respecté » (cadre vert) ou « infraction » (cadre rouge), accompagné d'une bannière d'alerte et d'un historique horodaté.

3.7 Résultats et évaluation

Sur la vidéo de test, le système se comporte de manière cohérente avec son objectif : le panneau Stop est détecté de façon stable, les véhicules sont correctement suivis et un verdict est rendu pour chaque véhicule concerné. La figure 3.5 synthétise les détections obtenues par classe d'objet ainsi que les verdicts.

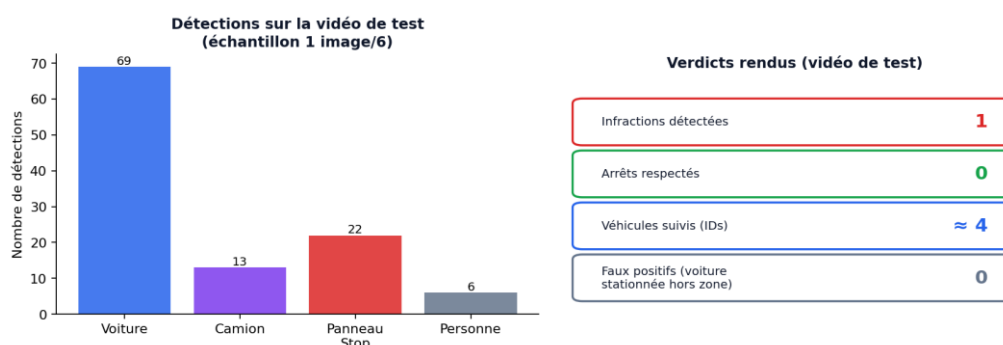


Figure 3.5 — Résultats de la détection et des verdicts

Deux enseignements se dégagent de cette évaluation. D'abord, le système détecte effectivement un comportement (l'arrêt ou son absence) et non un simple passage. Ensuite, la mesure d'immobilité, rapportée à la taille du véhicule, reste pertinente malgré les variations d'échelle des objets dans la scène. Une évaluation quantitative à grande échelle (précision, rappel) sur un jeu de vidéos annotées plus vaste constitue l'une des perspectives du projet.

Ces résultats demeurent ceux d'une démonstration sur une scène maîtrisée. Une évaluation quantitative à grande échelle (précision, rappel) nécessiterait un jeu de vidéos annotées plus vaste et varié ; elle constitue l'une des perspectives du projet.

3.8 Conclusion

Ce chapitre a détaillé la réalisation de TrafficGuard, de l'environnement de développement jusqu'à l'évaluation sur une vidéo réelle, en passant par les corrections qui ont fiabilisé la mesure d'arrêt. Le système atteint son objectif principal : juger un comportement, et non un simple objet.

Conclusion générale et perspectives

Ce projet de fin d'études a permis de concevoir et de réaliser une application web complète combinant intelligence artificielle et développement web. TrafficGuard démontre qu'il est possible, à l'aide d'outils libres et modernes, de construire un système capable non seulement de détecter des panneaux Stop et des véhicules, mais aussi d'évaluer un véritable comportement : l'arrêt à un panneau Stop.

L'apport central du projet réside dans la distinction, trop souvent négligée, entre détecter un objet et juger un comportement. En s'appuyant sur le suivi des véhicules et sur la mesure de leur mouvement dans le temps, le système rend un verdict fondé sur l'observation réelle de l'arrêt. Le travail de fiabilisation mené sur le moteur — critère d'immobilité invariant à l'échelle, zone de contrôle cohérente, jugement restreint à l'aire du Stop — a transformé une preuve de concept en un système dont la décision est géométriquement et physiquement justifiée.

Le développement a permis d'acquérir des compétences dans plusieurs domaines : la vision par ordinateur avec OpenCV, la détection et le suivi d'objets avec YOLOv8 et ByteTrack, le développement web avec Flask, la conteneurisation avec Docker, ainsi que la conduite d'un projet en équipe.

Le projet présente des limites assumées : la mesure de l'arrêt suppose une caméra fixe ; la détection est moins fiable de nuit ou par mauvais temps ; une occlusion prolongée peut faire perdre le suivi d'un véhicule ; et le modèle générique yolov8n n'est pas spécialisé pour le trafic. Ces limites tracent naturellement les perspectives d'évolution :

- Ré-entraîner le modèle de détection sur un jeu de données routier spécifique, afin d'améliorer la fiabilité dans des conditions variées.
- Constituer un jeu de vidéos annotées pour mener une évaluation quantitative (précision, rappel) à grande échelle.
- Intégrer la reconnaissance des plaques d'immatriculation pour identifier les véhicules en infraction.
- Générer automatiquement des rapports PDF récapitulatifs des infractions constatées.
- Compenser le mouvement de la caméra par stabilisation, pour s'affranchir de la contrainte de caméra fixe.
- Déployer le système sur un dispositif embarqué (par exemple Raspberry Pi) pour un usage en conditions réelles, et l'étendre à d'autres infractions (feu rouge, excès de vitesse).

TrafficGuard n'est pas seulement un travail académique : il constitue une base concrète et évolutive, susceptible d'être enrichie pour contribuer, à sa mesure, à l'amélioration de la sécurité routière.

Références

- [1] Ultralytics. YOLOv8 Documentation. Disponible sur : <https://docs.ultralytics.com>
- [2] Y. Zhang et al. ByteTrack : Multi-Object Tracking by Associating Every Detection Box. ECCV, 2022. Disponible sur : <https://github.com/ifzhang/ByteTrack>
- [3] J. Redmon, S. Divvala, R. Girshick, A. Farhadi. You Only Look Once : Unified, Real-Time Object Detection. CVPR, 2016.
- [4] OpenCV. OpenCV Documentation. Disponible sur : <https://docs.opencv.org>
- [5] NumPy Developers. NumPy Documentation. Disponible sur : <https://numpy.org/doc/>
- [6] Pallets Projects. Flask Documentation. Disponible sur : <https://flask.palletsprojects.com>
- [7] T.-Y. Lin et al. Microsoft COCO : Common Objects in Context. ECCV, 2014. Disponible sur : <https://cocodataset.org>
- [8] Docker Inc. Docker Documentation. Disponible sur : <https://docs.docker.com>
- [9] Python Software Foundation. Python 3 Documentation. Disponible sur : <https://docs.python.org/3/>
- [10] Cours de Vision par Ordinateur et d'Intelligence Artificielle, Licence Génie Informatique, 2025–2026.